TechTransit — Guía de entorno de desarrollo

> **Versión:** 1.0 (30-oct-2026). Entorno local con la base de datos PostgreSQL en Docker.

1. Resumen

Componente	Tecnología	Puerto local
Base de datos	PostgreSQL 17 (contenedor Docker)	**5433** → 5432 del contenedor
Backend (API)	Node.js 24 + Express 5 + Prisma 7 + Zod 4 + TypeScript 5.9	**4000**
Frontend	React 19 + Vite 8 + Tailwind 4 + React Router 7	**3000**

El frontend llama a `/api/v1/...`; en desarrollo Vite redirige esas peticiones al backend en `localhost:4000`, así que no hace falta configurar CORS ni URL del API.

¿Por qué 5433 y no 5432? En muchas máquinas el 5432 ya lo ocupa otro Postgres (local o de otro proyecto en Docker). Usar 5433 evita el conflicto. Se puede cambiar con la variable `DB_PORT`.

2. Requisitos previos

Herramienta	Versión	Comprobar
Node.js	24.x (la misma que usa CI)	`node -v`
npm	11.x	`npm -v`
Docker + Docker Compose v2	Docker 24+	`docker compose version`
DBeaver (opcional)	Community 24+	—
Git	cualquiera reciente	`git --version`

En Windows se recomienda trabajar dentro de **WSL2** con Docker Desktop (integración WSL activada) y clonar el repo dentro del sistema de archivos de Linux (`~/...`), no en `/mnt/c/...`.

3. Puesta en marcha (primera vez)

3.1 Clonar e instalar dependencias

```
git clone <url-del-repo> techtransit
cd techtransit
npm ci            # instala frontend + backend (npm workspaces)
```

3.2 Levantar la base de datos

```
npm run db:up     # docker compose up -d --wait db
```

Esto crea el contenedor `techtransit-db` con:

Parámetro	Valor por defecto
Host	`localhost`
Puerto	`5433`
Base de datos	`techtransit`
Usuario	`techtransit`
Contraseña	`techtransit`

Los datos se guardan en el volumen Docker `techtransit_pgdata`, así que **sobreviven** a reinicios y a `npm run db:down`.

Comprobar que está sano:

```
docker compose ps          # STATUS debe decir "healthy"
```

3.3 Configurar variables de entorno del backend

```
cp backend/.env.development.example backend/.env
```

> Si ya tienes un `backend/.env` apuntando a Supabase, respáldalo primero (`mv backend/.env backend/.env.supabase`).

Contenido relevante (ya apunta al contenedor):

```
PORT=4000
NODE_ENV=development
FRONTEND_URL=http://localhost:3000
LOG_LEVEL=debug

DATABASE_URL="postgresql://techtransit:techtransit@localhost:5433/techtransit"
DIRECT_URL="postgresql://techtransit:techtransit@localhost:5433/techtransit"

JWT_ACCESS_SECRET=dev_access_secret_cambiar_minimo_32_caracteres
JWT_REFRESH_SECRET=dev_refresh_secret_cambiar_minimo_32_caracteres
JWT_ACCESS_EXPIRY=15m
JWT_REFRESH_EXPIRY=7d
```

> En local `DATABASE_URL` y `DIRECT_URL` son iguales. Solo difieren en Supabase (pooler 6543 vs conexión directa 5432). Los secretos JWT deben tener **mínimo 32 caracteres**; el backend no arranca si no se cumple.

El frontend **no necesita** `.env` en desarrollo.

3.4 Crear tablas y datos de prueba

```
npm run db:setup -w backend
```

Equivale a `prisma migrate deploy` (crea las tablas) + `prisma generate` (genera el cliente en `backend/src/generated/prisma/`, carpeta ignorada por git) + `prisma db seed` (usuarios de prueba).

Usuarios que crea el seed:

Rol	Email	Contraseña
ADMIN	admin@techtransit.dev	Admin1234!
SUPERVISOR	supervisor@techtransit.dev	Super1234!
COLABORADOR	colaborador@techtransit.dev	Colab1234!

3.5 Arrancar la aplicación

En dos terminales:

```
# Terminal 1 — API con recarga automática
npm run start:dev -w backend

# Terminal 2 — Frontend
npm run dev -w frontend
```

- Frontend: http://localhost:3000
- Health check del API: http://localhost:4000/api/v1/health → `{"status":"ok",...}`

---

4. Conectarse a la base de datos con DBeaver

1. **Database → New Database Connection → PostgreSQL → Next.**
2. Pestaña **Main**:

Campo	Valor
Connect by	Host
Host	`localhost`
Port	`5433`
Database	`techtransit`
Authentication	Database Native
Username	`techtransit`
Password	`techtransit` (marcar *Save password*)

3. Si DBeaver lo pide, aceptar **Download driver files**.
4. **Test Connection** → debe decir *Connected*. **Finish.**
5. Las tablas están en `techtransit → Schemas → public → Tables`.

Tablas esperadas: `entidad`, `usuario`, `aliado`, `proceso_precontractual`, `proceso_aliado`, `historial_proceso`, `contrato`, `hito_contrato` y `_prisma_migrations` (control interno de Prisma, no editar).

Notas:

- En WSL2 con Docker Desktop, `localhost` desde Windows llega al contenedor sin configuración extra.
- Los enums se llaman `"RolUsuario"`, `"EstadoProceso"`, etc. (con comillas, sensibles a mayúsculas) porque los genera Prisma.
- Alternativa sin DBeaver: `npm run db:studio -w backend` abre Prisma Studio en el navegador.

---

5. Comandos del día a día

Acción	Comando
Levantar BD	`npm run db:up`
Detener BD (conserva datos)	`npm run db:down`
Ver logs de la BD	`npm run db:logs`
Reiniciar BD desde cero (**borra datos**)	`npm run db:reset` y luego `npm run db:setup -w backend`
Consola `psql` dentro del contenedor	`docker exec -it techtransit-db psql -U techtransit`
Nueva migración tras editar `schema.prisma`	`npm run db:migrate -w backend -- --name <descripcion>`
Regenerar cliente Prisma	`npm run db:generate -w backend`
Volver a cargar usuarios de prueba	`npm run db:seed -w backend`
Lint (todo)	`npm run lint`
Build (todo)	`npm run build`
Formatear código	`npm run format`

Flujo para cambiar el modelo de datos

1. Editar `backend/prisma/schema.prisma`.
2. `npm run db:migrate -w backend -- --name agrega_campo_x` → crea `prisma/migrations/<fecha>_agrega_campo_x/`, la aplica y regenera el cliente.
3. Hacer commit del `schema.prisma` **y** de la carpeta de migración nueva.

> **Prisma 7:** `migrate dev` ya no ejecuta el seed automáticamente; si se necesita, correr `npm run db:seed -w backend`. La configuración del CLI (URL para migraciones, comando de seed) está en `backend/prisma.config.ts`; en tiempo de ejecución el backend se conecta con `@prisma/adapter-pg` (`backend/src/db/prisma.ts`).

> La fuente de verdad del esquema es `backend/prisma/schema.prisma` + `backend/prisma/migrations/`. El archivo `database/schema.sql` es histórico y **no** coincide con la base real; no usarlo para crear la BD.

---

6. Estructura del repositorio

```
techtransit/
├── backend/                 API Express
│   ├── prisma.config.ts     configuración del CLI de Prisma
│   ├── prisma/              schema.prisma, migraciones, seed.ts
│   └── src/
│       ├── generated/       cliente Prisma generado (no versionado)
│       ├── config/          env (validado con Zod), logger (pino)
│       ├── middleware/      auth JWT, roles, errores
│       └── modules/         auth, dashboard, procesos, contratos,
│                            entidades, aliados, usuarios, notificaciones
├── frontend/                SPA React + Vite
│   └── src/                 pages, components, lib (clientes API), store (zustand)
├── database/schema.sql      (histórico, ver nota en §5)
├── docs/                    documentación
├── docker-compose.yml       Postgres local
├── render.yaml              despliegue en Render
└── .github/workflows/ci.yml lint + build en cada PR
```

7. Solución de problemas

Síntoma	Causa probable	Solución
`port is already allocated` al hacer `db:up`	Algo ya usa el 5433	Usar otro puerto: `DB_PORT=5434 npm run db:up` y ajustar `DATABASE_URL` / `DIRECT_URL` y DBeaver
Backend termina con `Variables de entorno inválidas`	Falta `backend/.env` o secretos < 32 caracteres	Revisar §3.3
`P1001: Can't reach database server`	El contenedor no está corriendo o puerto distinto	`docker compose ps`; verificar el puerto en `DATABASE_URL`
`Port 3000 is already in use` en Vite	Otro proyecto usa el 3000	Detener el otro proceso o usar otro puerto: `VITE_PORT=3100 npm run dev -w frontend` **y** poner `FRONTEND_URL=http://localhost:3100` en `backend/.env`
Login devuelve 429	Rate limit de auth (20 intentos / 15 min)	Esperar o reiniciar el backend
DBeaver: `password authentication failed`	El volumen se creó con otra contraseña	`npm run db:reset` (borra datos) y volver a migrar + seed
`Cannot find module '../generated/prisma/client'`	No se ha generado el cliente Prisma	`npm run db:generate -w backend`
Errores de tipos de Prisma tras `git pull`	Cliente desactualizado	`npm run db:generate -w backend`
Pantalla en blanco con `Invalid hook call`	Quedó una copia vieja de React o la caché de Vite	`rm -rf frontend/node_modules/.vite && npm ci`

8. Entornos

Entorno	Base de datos	Configuración
Desarrollo local	Postgres 17 en Docker	`backend/.env` (este documento)
CI (GitHub Actions)	No usa BD (genera cliente Prisma + lint + build)	`.github/workflows/ci.yml`
Producción	Supabase (PostgreSQL)	Variables en el dashboard de Render (`render.yaml`)

9. Referencia de configuración

Variable	Dónde	Uso
`DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`	entorno de la shell al usar `docker compose`	Sobrescriben los valores por defecto del contenedor
`DATABASE_URL`	`backend/.env`	Conexión del backend en tiempo de ejecución
`DIRECT_URL`	`backend/.env`	Conexión para migraciones (si falta se usa `DATABASE_URL`)
`VITE_PORT`	entorno al ejecutar `npm run dev -w frontend`	Puerto del servidor de Vite (por defecto 3000)
`VITE_API_PROXY_TARGET`	entorno al ejecutar `npm run dev -w frontend`	Destino del proxy `/api` (por defecto `http://localhost:4000`)
`VITE_API_BASE_URL`	build del frontend (solo producción)	URL absoluta del API; vacío = `/api/v1`